

DATA VORTEX ASSESSMENT: PHASE 2 ANALYTICAL INSIGHT REPORT

Relational Database Core & Advanced Querying

Author: Himanshu Gaur | Execution Environment: Jupyter Notebook & SQLite (social_engine.db)

1. Executive Summary & Framework Overview

This report establishes the core data infrastructure and analytical outcomes for Phase 2 of the Data Vortex project. Building directly on top of the clean, median-imputed records established previously, all datasets were successfully migrated into a relational SQLite database architecture (social_engine.db). Within this environment, three analytical perspectives were deployed: temporal trend detection, behavioral segmentation of high-value participants, and rigorous outlier discovery using statistical thresholding.

2. Relational Core & Ingestion Framework

To transition from flat-file formats into a structured relational setup, Python's pandas library interacted directly with SQLite via programmatic table creation (users and posts). This ensured structural integrity, zero-null constraints, and full reproducibility across all operational scripts.

3. Core SQL Queries & Analytical Findings

Query 1: Temporal Trend Detection (Monthly Platform Performance)

Objective: Monitor monthly engagement trajectories and platform dominance over time.

SQL Logic & Implementation:

```
SELECT
    strftime('%Y-%m', timestamp) AS post_month,
    platform,
    COUNT(post_id) AS total_posts,
    SUM(likes) AS total_likes,
    ROUND(AVG(likes), 2) AS avg_likes_per_post
FROM posts
WHERE timestamp IS NOT NULL
GROUP BY post_month, platform
ORDER BY post_month DESC, total_likes DESC;
```

Analytical Insights: Activity spans consistently across 2024 and 2025. Platforms like Reddit and YouTube consistently secure top-tier cumulative interactions (e.g., Reddit peaking at 28,181 likes in December 2025).

Query 2: Behavioral Grouping (Top Active Users)

Objective: Isolate high-impact content creators and platform influencers based on aggregate interaction metrics.

SQL Logic & Implementation:

```
SELECT
    p.user_id,
    COUNT(p.post_id) AS total_posts_made,
    SUM(p.likes + p.shares + p.comments) AS cumulative_engagement,
    ROUND(AVG(p.likes), 2) AS avg_likes_per_post
FROM posts p
GROUP BY p.user_id
ORDER BY cumulative_engagement DESC
LIMIT 10;
```

Analytical Insights: Segmenting core creators isolates power users such as `user_zqv2vrf5`, who leads the network with 22 posts and 90,374.5 total engagement units.

Query 3: Anomaly Discovery (Viral Outlier Identification)

Objective: Detect posts exhibiting extreme engagement levels that deviate significantly from the baseline distribution.

SQL Logic & Implementation:

```
SELECT
    post_id,
    user_id,
    platform,
    likes,
    shares,
    comments,
    timestamp
FROM posts
WHERE likes > (
    SELECT AVG(likes) + (3 * (SELECT AVG((likes - (SELECT AVG(likes) FROM
posts))*(likes - (SELECT AVG(likes) FROM posts))) FROM posts))
    FROM posts
)
ORDER BY likes DESC;
```

Analytical Insights: Establishes a mathematically sound boundary for viral outlier isolation, preventing skewness in subsequent modeling steps.

4. Visualizations & Performance Distributions

- **Figure 5.1 (Platform Benchmarks):** Highlights Reddit as the top-performing medium with an average of 3,522.6 likes per post.

- **Figure 5.2 (User Leaderboard):** Maps the top 5 creators ordered by combined cumulative interactions.